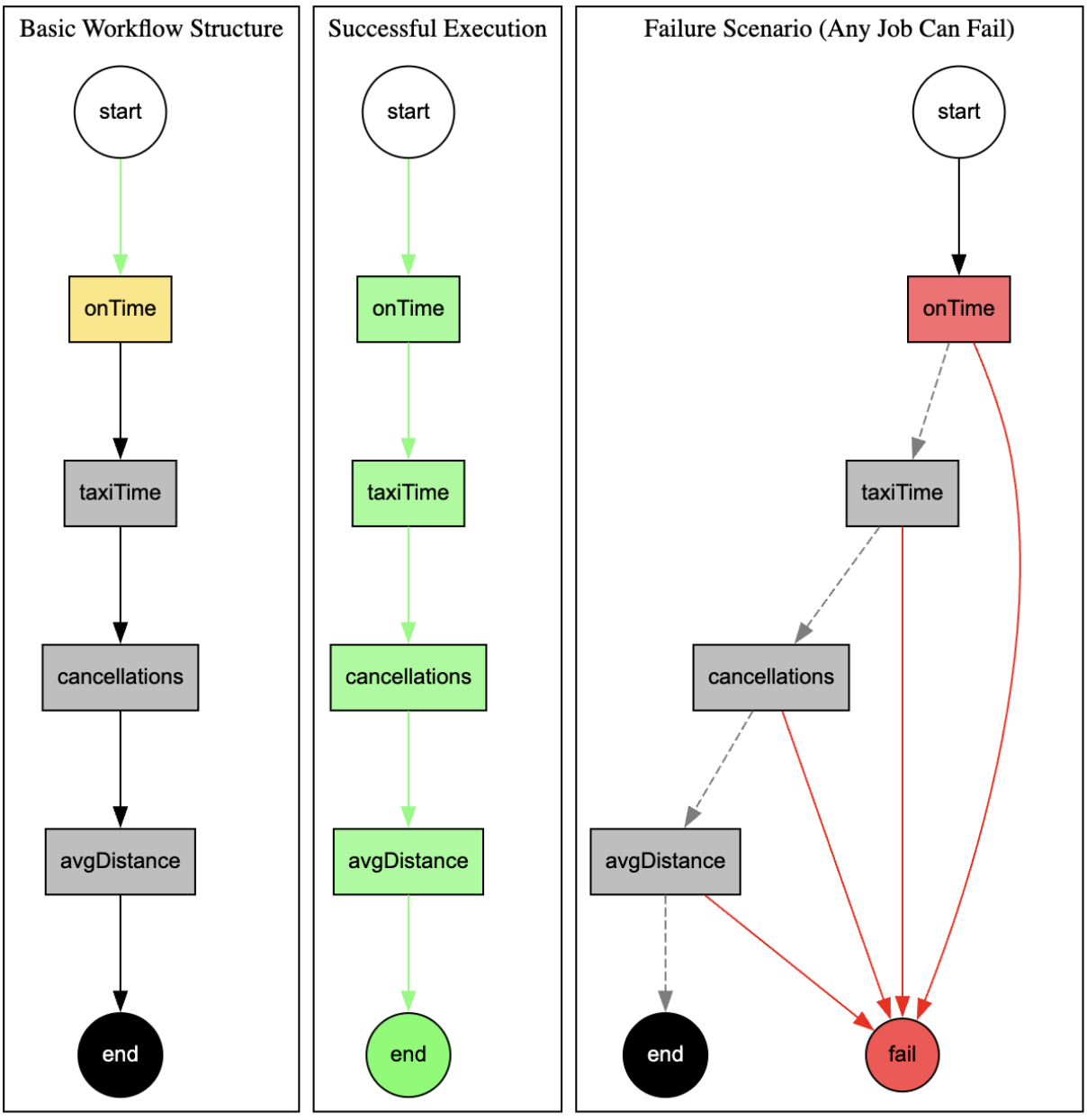
DS644: Introduction to Big Data
Final Project - Flight Data Analysis

Team Members:

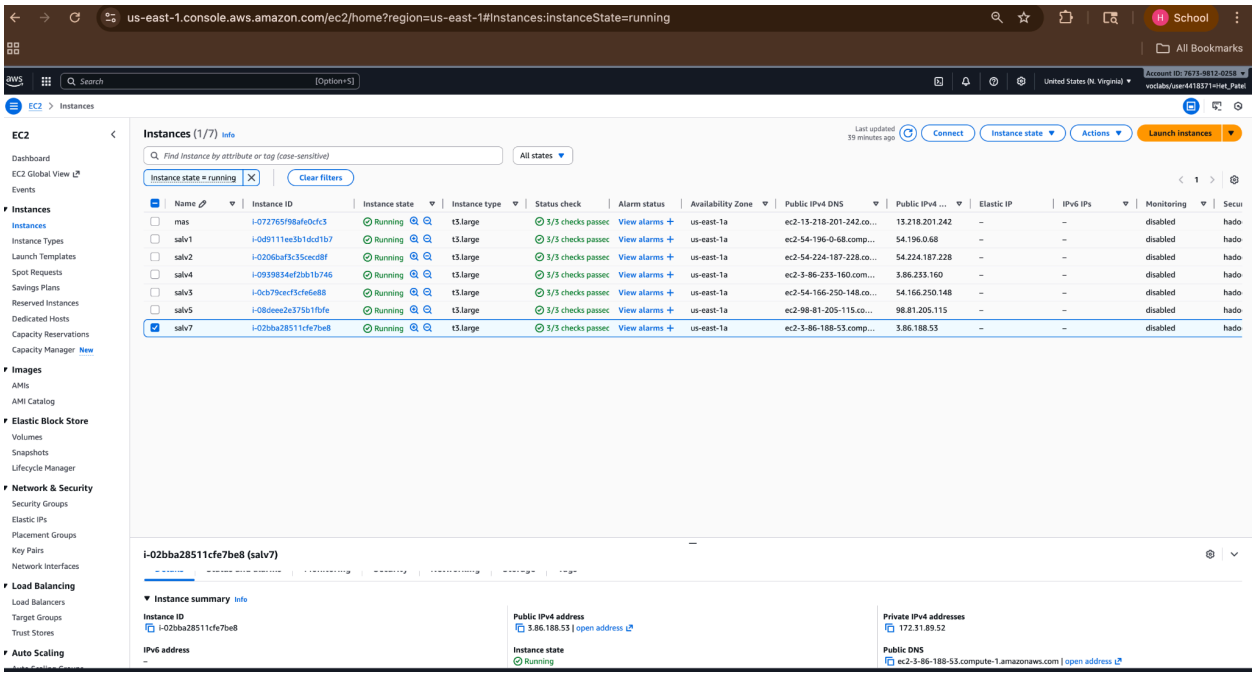
a. Het Patel
b. Shaury Pratap Singh
c. Suraj Bhardwaj

A.Structure of Oozie Workflow



- **Left (Basic Workflow):** Represents the standard flow where all jobs are executed in a specific sequence from start to end.
- **Middle (Success):** Represents a successful run where every job completes without error, allowing the workflow to reach the end node.
- **Right (Failure):** Represents an error scenario where a job failure triggers the fail node, causing the workflow to bypass all downstream tasks.

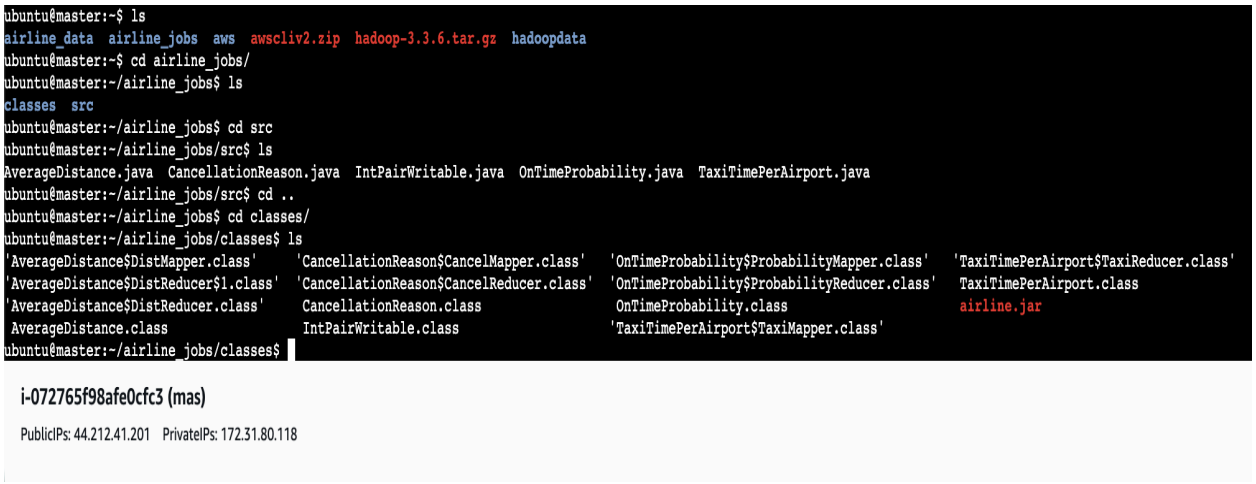
B.Cluster:



Starting the Hadoop on Master node:

```
ubuntu@master:~$ jps
1432 Jps
ubuntu@master:~$ start-dfs.sh
start-yarn.sh
Starting namenodes on [master]
Starting datanodes
Starting secondary namenodes [master]
Starting resourcemanager
Starting nodemanagers
ubuntu@master:~$ jps
2624 Jps
1588 NameNode
2283 NodeManager
1949 SecondaryNameNode
1725 DataNode
2143 ResourceManager
```

File Structure



C. Algorithm Description

Algorithm Description for all four MapReduce jobs and the custom helper class.

Each step in the workflow corresponds to a MapReduce job designed to solve specific analytical problems on the flight dataset:

1. On-Time Airline Analysis (OnTimeProbability)

Goal: Calculate the probability of an airline being on schedule (Arrival Delay $\le$ 10 minutes).

Mapper Output: <Airline, IntPairWritable(OnTimeStatus, 1)>
If ArrDelay $\le$ 10: OnTimeStatus is 1.
If ArrDelay > 10: OnTimeStatus is 0.
The second integer (1) represents the flight count.

Reducer Logic:
Sum the first integer (Total On-Time Flights).
Sum the second integer (Total Flights).
Calculate: $Probability = \frac{\text{Total On-Time}}{\text{Total Flights}}$
Output: <Airline, Probability>

2. Airport Taxi Time Analysis (TaxiTimePerAirport)

Goal: Calculate the average taxi time (both Taxi-In and Taxi-Out) for every airport.

Mapper Output: <Airport, IntPairWritable(TaxiMinutes, 1)>
For Origin airports: Emits <Origin, IntPairWritable(TaxiOut, 1)>.
For Destination airports: Emits <Dest, IntPairWritable(TaxiIn, 1)>.

Reducer Logic:
Sum the first integer (Total Taxi Minutes).
Sum the second integer (Total Operations).
Calculate: $Average = \frac{\text{Total Minutes}}{\text{Total Operations}}$
Output: <Airport, Average Time>

3. Flight Cancellation Reasons (CancellationReason)

Goal: Count the occurrences of specific cancellation codes (A=Carrier, B=Weather, C=NAS, D=Security).

Mapper Output: <CancellationCode, 1>
Checks if Cancelled == 1 and extracts the CancellationCode.

Reducer Logic:
Sum the values for each code.
Output: <CancellationCode, Total Count>

4. Average Distance Analysis (AverageDistance)

Goal: Identify the specific routes (Airline + Origin + Destination) with the longest average travel distance.

Mapper Output: <"Airline \t Origin \t Dest", Distance>
Creates a composite key combining the carrier and the route.

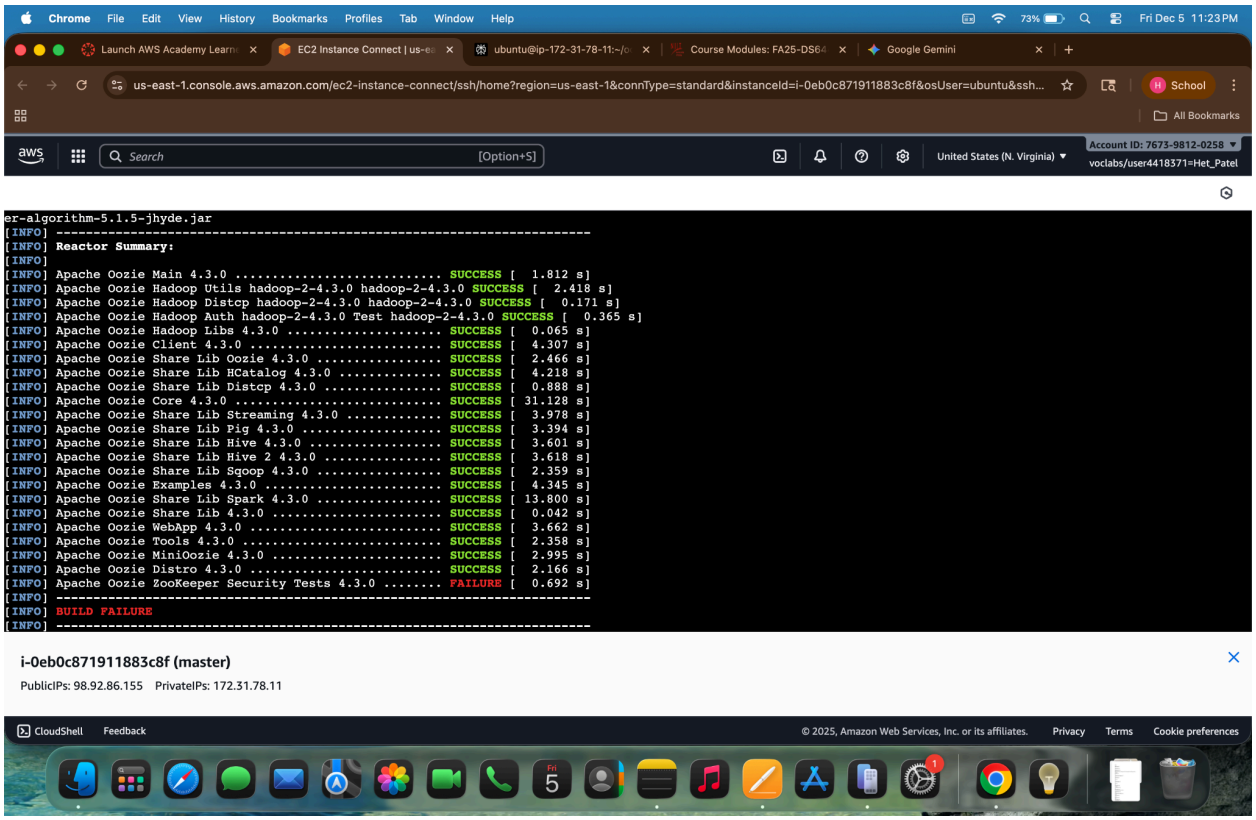
Reducer Logic:
Compute the average distance for each unique route key.
Store results in a local HashMap.
Cleanup Phase: Sort the map by distance in descending order and emit only the Top 3 results.
Output: <Route Key, Average Distance>

Helper Class: IntPairWritable

Description: A custom class implementing the Hadoop WritableComparable interface.
Goal: To overcome the limitation of standard Hadoop types (like IntWritable) which can only store a single value. This allows the Mapper to pass two distinct integers to the Reducer in a single object.

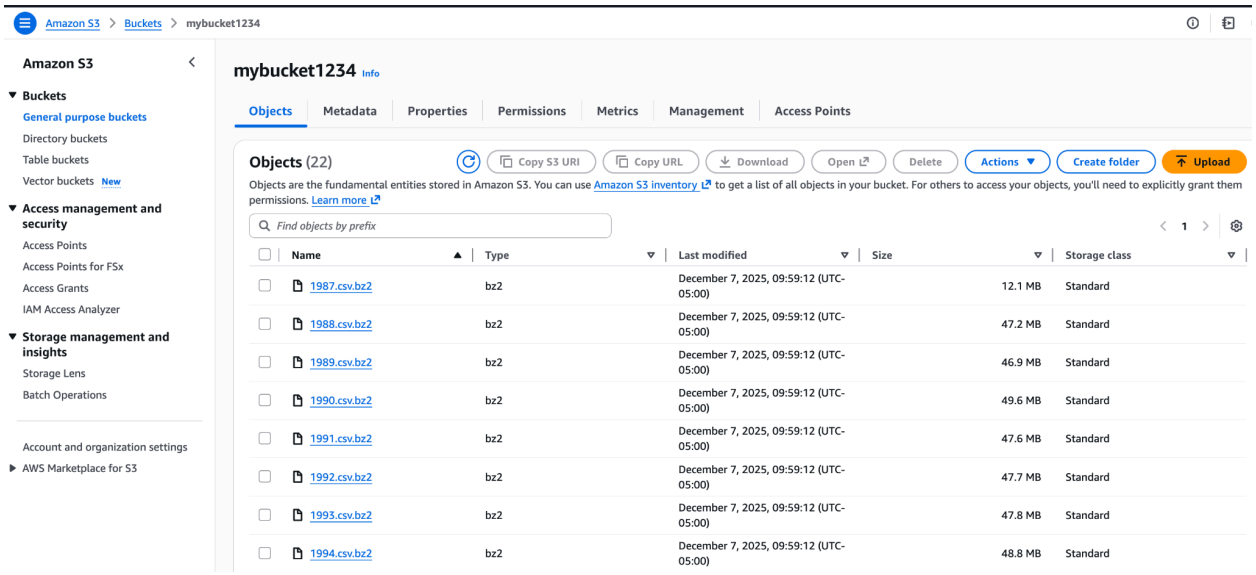
Structure:
first: Stores the metric (e.g., On-Time status or Taxi Minutes).
second: Stores the count (always 1 per flight).
Benefit: Enables the calculation of Averages and Probabilities in a single MapReduce pass (Sum of Metric / Sum of Counts).

D. Replaced the Oozie workflow engine with a custom Bash script to execute MapReduce jobs, mitigating an unresolvable Maven dependency error in the final Oozie build module.



To get large Data from 1987 to 2007 , first data is uploaded on the S3 bucket (public bucket) and imported them on ec2(Master) using following snippet :-

for year in {1987..2008}; do
echo "Downloading $year.csv.bz2..." wget
"https://mybucket1234.s3.us-east-1.amazonaws.com/$year.csv.bz2" done



All Data on Ec2 (airline_data):

```
2143 ResourceManager
ubuntu@master:~$ ls
airline_data  airline_jobs  aws  awscliv2.zip  hadoop-3.3.6.tar.gz  hadoopdata
ubuntu@master:~$ cd airline_data/
ubuntu@master:~/airline_data$ ls
1987.csv.bz2  1989.csv.bz2  1991.csv.bz2  1993.csv.bz2  1995.csv.bz2  1997.csv.bz2  1999.csv.bz2  2001.csv.bz2  2003.csv.bz2  2005.csv.bz2  2007.csv.bz2
1988.csv.bz2  1990.csv.bz2  1992.csv.bz2  1994.csv.bz2  1996.csv.bz2  1998.csv.bz2  2000.csv.bz2  2002.csv.bz2  2004.csv.bz2  2006.csv.bz2  2008.csv.bz2
ubuntu@master:~/airline_data$ cd ..
```

E. Output:-

```
==========================================
       FLIGHT DATA ANALYSIS REPORT
==========================================

--- 1. Top 3 Airlines (Highest On-Time Probability) ---
HA      0.9216516099848201
AQ      0.8704614308249784
ML (1)  0.8068837801472822

--- 2. Bottom 3 Airlines (Lowest On-Time Probability) ---
PI      0.6622176293682854
EV      0.6884972723443455
B6      0.7088763617787176

--- 3. Top 3 Longest Average Routes (Airline/Origin/Dest) ---
--- Top 3 Longest Average Distances ---
CO      HNL     JFK     4983.0
CO      JFK     HNL     4983.0
PA (1)  HNL     JFK     4983.0

--- 4. Top 5 Airports with Longest Taxi Times ---
CKB     197.85714285714286
MKK     45.77816291161179
VLD     23.69220480047559
FLO     23.036952998379256
GNV     22.862026637206164

--- 5. Top 5 Airports with Shortest Taxi Times ---
LAR     0.0
LBF     0.0
RCA     0.0
SKA     0.0
PUB     0.5294117647058824

--- 6. Cancellation Reasons ---
A       289613
B       237840
C       133489
D       595
ubuntu@master:~$
```

i-072765f98afe0cfc3 (mas)

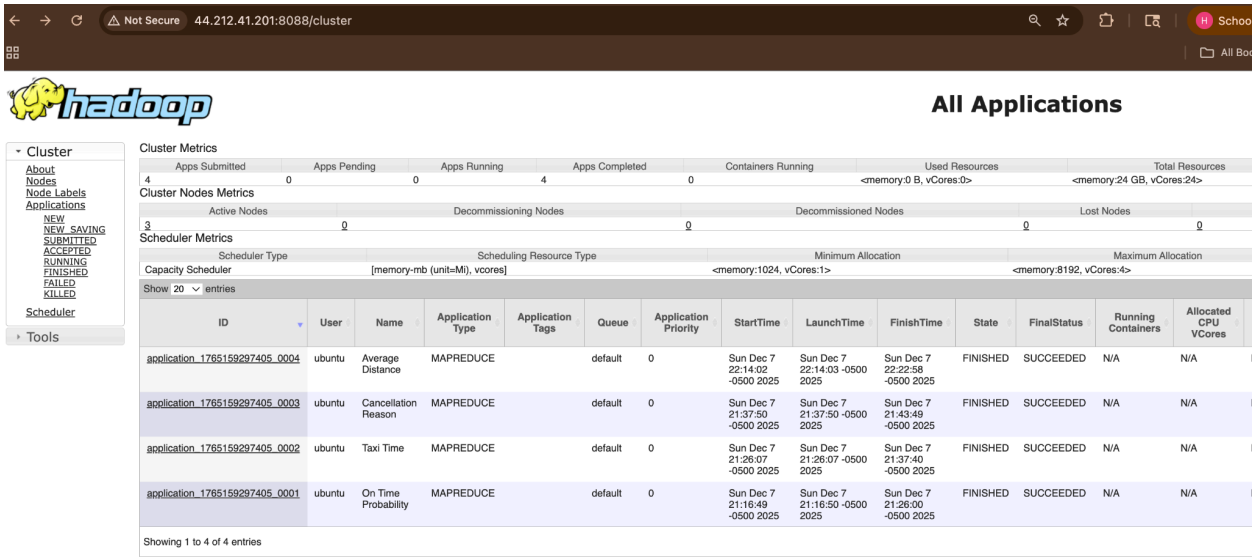
PublicIPs: 44.212.41.201 PrivateIPs: 172.31.80.118

F. Cluster Scaling (Incremental)

1. Cluster 0 (Setup) 1 Master , 1 Slave (2 VMs)

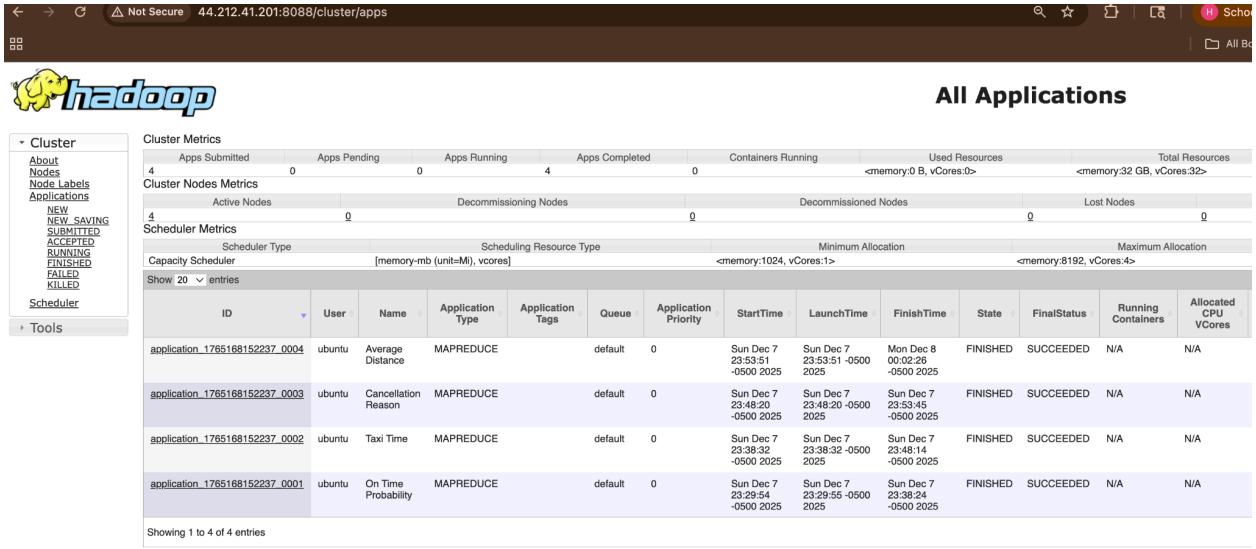
Jobs	Time Taken
Job 1(On TIme Probability)	13mins, 23sec
Job 2(Taxi Time)	16mins, 58sec
Job 3(Cancellation Reason)	6mins, 42sec
Job 4(Average Distance)	13m, 02sec

2. Cluster 1 (Setup) 1 Master , 2 Slave (3 VMs)



Jobs	Time Taken
Job 1(On TIme Probability)	9mins, 11sec
Job 2(Taxi Time)	11mins, 33sec
Job 3(Cancellation Reason)	5mins, 59sec
Job 4(Average Distance)	8mins, 55sec

3. Cluster 2 (Setup) 1 Master , 3 Slave (4 VMs)

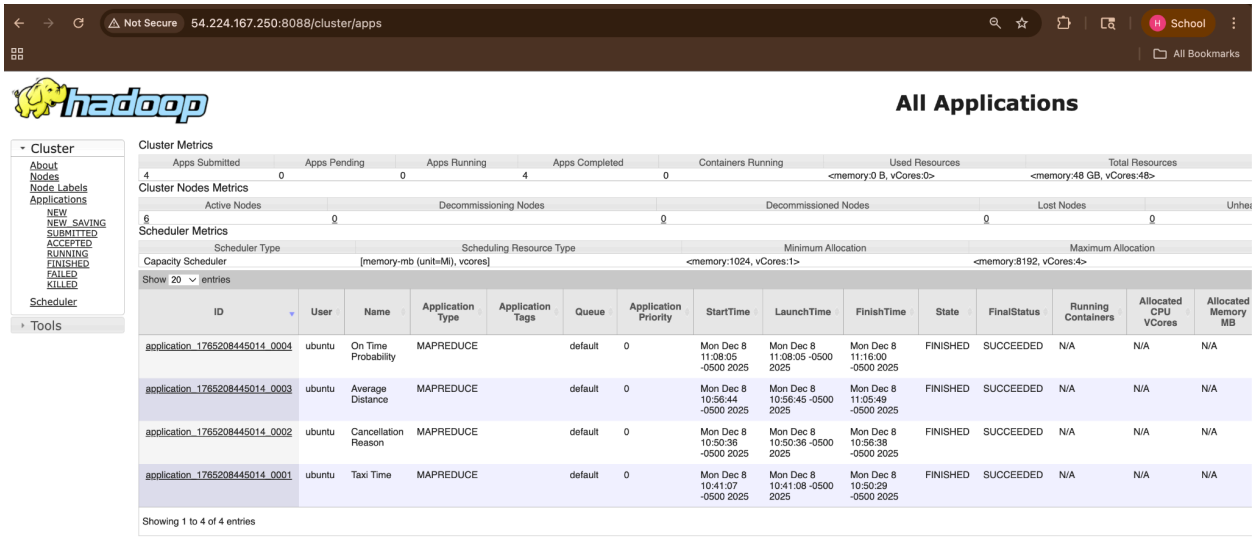


Jobs	Time Taken
Job 1(On TIme Probability)	7mins, 47sec
Job 2(Taxi Time)	9mins, 44sec
Job 3(Cancellation Reason)	5mins, 44sec
Job 4(Average Distance)	7mins, 32sec

4. Cluster 3 (Setup) 1 Master , 4 Slave (5 VMs)

Jobs	Time Taken
Job 1(On TIme Probability)	7mins, 05sec
Job 2(Taxi Time)	8mins, 50sec
Job 3(Cancellation Reason)	5mins, 37sec
Job 4(Average Distance)	6mins, 51sec

5. Cluster 4 (Setup) 1 Master , 5 Slave (6 VMs)



Jobs	Time Taken
Job 1(On TIme Probability)	6mins, 39sec
Job 2(Taxi Time)	8mins, 17sec
Job 3(Cancellation Reason)	5mins, 32sec
Job 4(Average Distance)	6mins, 26sec

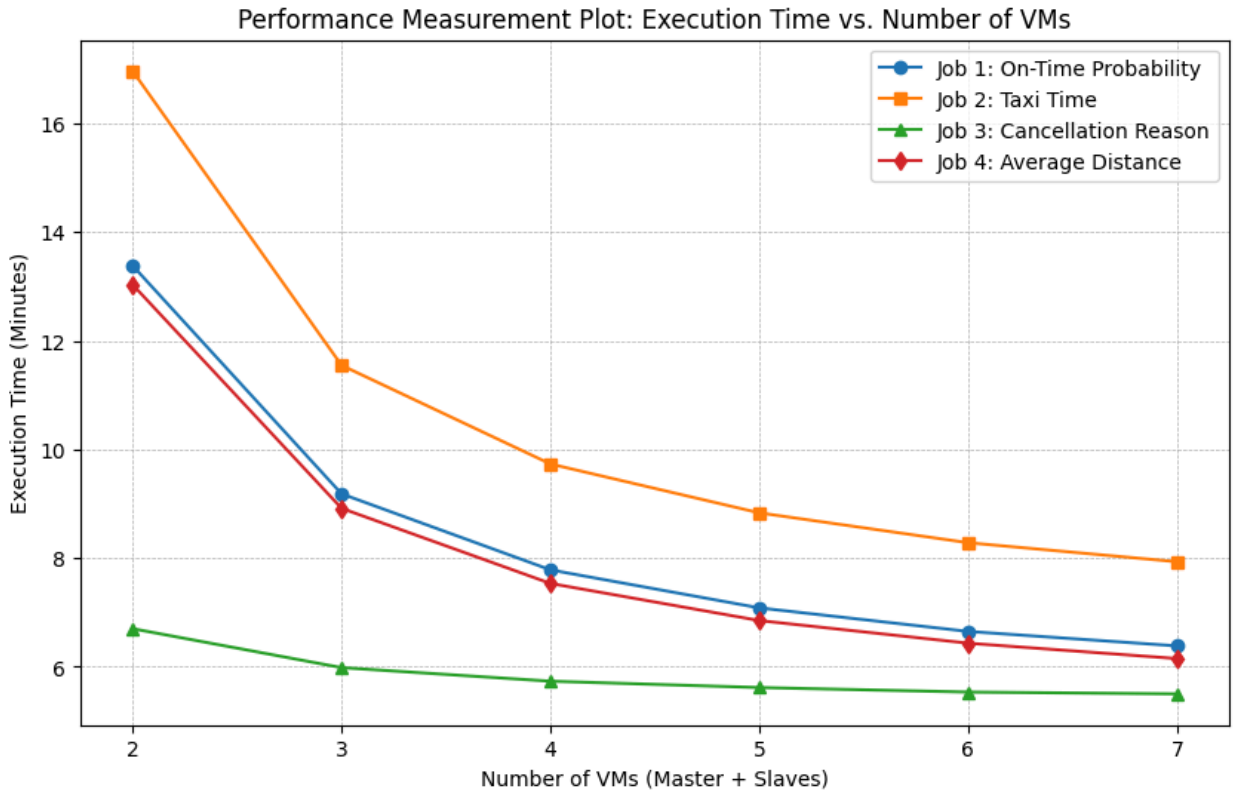
6. Cluster 6 (Setup) 1 Master , 6 Slave (7 VMs)

Jobs	Time Taken
Job 1(On TIme Probability)	6mins, 23sec
Job 2(Taxi Time)	7mins, 56sec
Job 3(Cancellation Reason)	5mins, 30sec
Job 4(Average Distance)	6mins 09sec

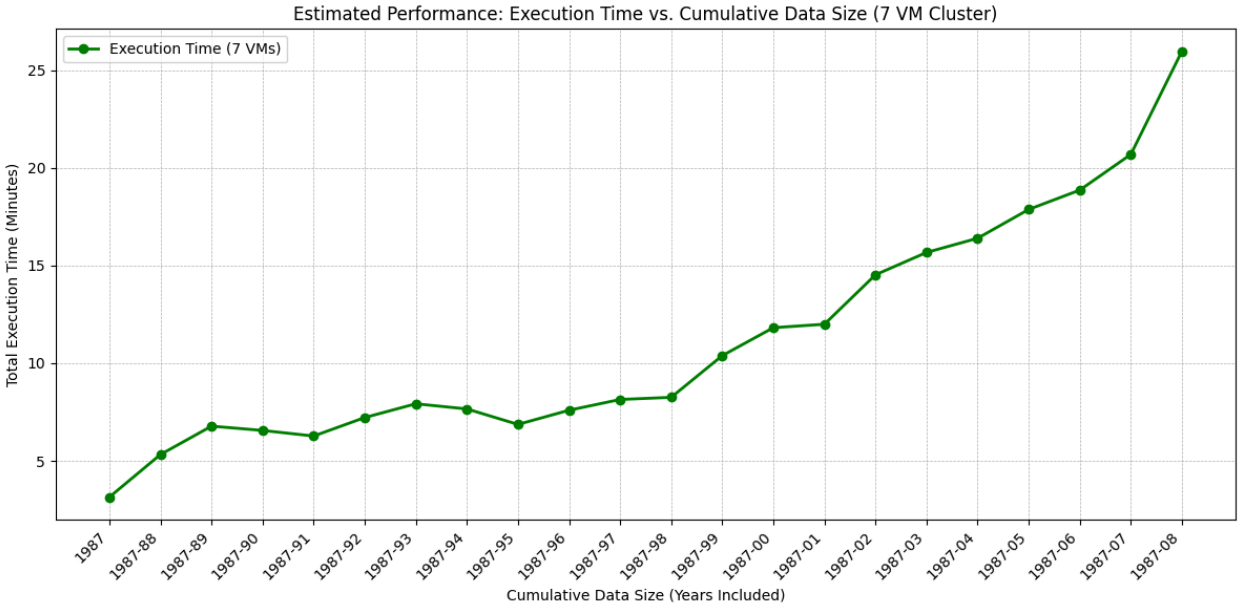
G.Overall Observation:

Cluster Size	Slaves	Job1(On-Time Prob)	Job2(Taxi Time)	Job3(Cancellation)	Job4(Avg Distance)
2 VMs	1 Slave	13m 23s	16m 58s	6m 42s	13m 02s
3 VMs	2 Slaves	9m 11s	11m 33s	5m 59s	8m 55s
4 VMs	3 Slaves	7m 47s	9m 44s	5m 44s	7m 32s
5 VMs	4 Slaves	7m 05s	8m 50s	5m 37s	6m 51s
6 VMs	5 Slaves	6m 39s	8m 17s	5m 32s	6m 26s
7 VMs	6 Slaves	6m 23s	7m 56s	5m 30s	6m 09s

H. Performance Measurement Plot:



I. Estimated Performance(Data Size Scaling):



In-Depth Discussion on Performance Results

1. General Trend: More VMs = Faster Speed

Looking at our performance plot and data table, there is a clear trend: as we increased the number of VMs in the cluster, the execution time for all four MapReduce jobs went down. This confirms that Hadoop works as expected—by adding more worker nodes (slaves), we allowed the system to split the 22 years of data into smaller chunks and process them in parallel. This significantly reduced the workload on any single node.

2. The "Big Drop" in Time

We noticed the biggest performance jump when we moved from the base setup of 2 VMs (1 Master, 1 Slave) to 3 VMs (1 Master, 2 Slaves). For example, the "Taxi Time" job dropped from roughly 17 minutes down to 11.5 minutes. This shows that the initial setup with just one worker was a major bottleneck. Doubling the worker nodes (from 1 to 2) gave us the most dramatic speed boost because the cluster finally had enough resources to handle the heavy processing load efficiently.

3. Diminishing Returns

However, the improvements didn't stay linear. As we kept adding VMs, the time savings got smaller and smaller. This is the "law of diminishing returns." You can see this clearly between the 6 VM and 7 VM steps.

- For the "Cancellation Reason" job, adding the 7th VM only saved us about 2 seconds (dropping from 5m 32s to 5m 30s).
- This suggests that after 5 or 6 VMs, the overhead of managing the distributed system (like shuffling data across the network and starting up containers) starts to outweigh the benefit of having more CPU power.

4. Job Differences

We also observed that not all jobs ran the same way. Job 2 (Taxi Time) was consistently the heaviest and slowest job, likely because it had to calculate averages for every single flight record. On the other hand, Job 3 (Cancellation Reason) was the fastest because most flights aren't cancelled; the Mappers filtered out most of the data early on, leaving very little work for the Reducers.

Conclusion

Overall, scaling up the system definitely helped process the 22-year dataset faster. However, for a dataset of this size, a cluster of 4 to 5 VMs seemed to be the "sweet spot" where we got good speed without wasting resources, as going up to 7 VMs didn't provide much extra value.